Cash Compass

A privacy-first personal finance companion for Android.
Built in Flutter. Everything on device.

TECHNICAL REPORT

Version 0.2.0 (build 9) · Android, minSdk 24 / targetSdk 36

Vansh · Kintan · Athena · Akshit · Meehika

What the app does

Cash Compass is an Android application for tracking day-to-day money: what came in, what went out, what is safe to spend today, and how close each savings goal is. It is built for students and early-career users, where the interesting question is rarely "what is my net worth" and almost always "can I afford this right now".

The defining constraint is that **there is no server**. No account is required, no financial record leaves the handset, and the app is fully usable with the network off. Two things reach the internet at all: the daily exchange-rate lookup and the webfont on first launch. Receipt scanning, which is the heaviest feature in the product, runs its optical character recognition entirely on-device.

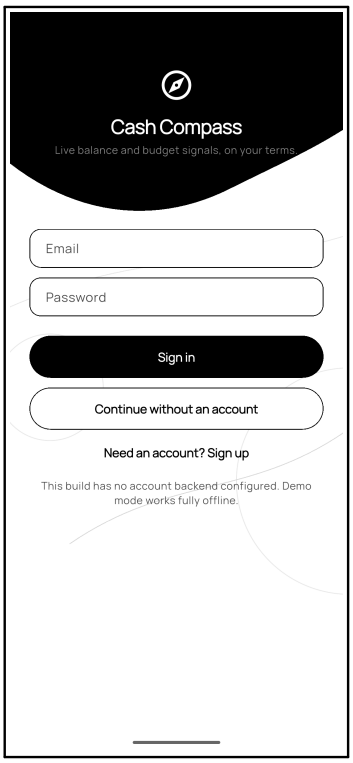
23,436	537	1,044	16
LINES OF DART	TESTS PASSING	TRANSLATED STRINGS	WORKSPACE WIDGETS

At a glance

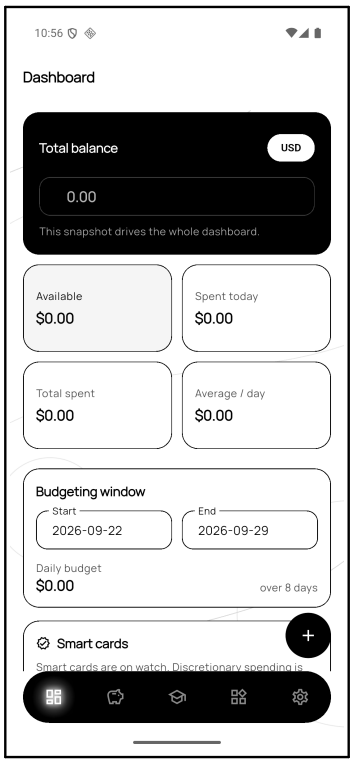
Area	Detail
Platform	Android only. Flutter / Dart, Material 3, minSdk 24, targetSdk 36.
Architecture	Three layers - pure-Dart logic, ChangeNotifier state, widget tree - bridged by a presenter layer that turns rules into words.
Persistence	shared_preferences, one JSON blob per domain, debounced writes flushed on app pause.
Currencies	USD, INR and RUB, with live rates. All amounts stored in USD and converted only at the display boundary.
Languages	English and Russian, complete - 522 keys each, including plural forms that decline correctly in Russian.
Offline	Complete. Demo mode needs no account and no connection.

The product, screen by screen

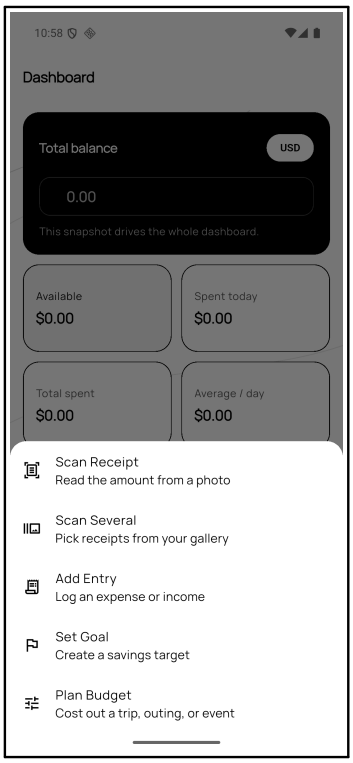
Five tabs carry the day-to-day work; two further screens - Financial Tips and Achievements - open from Settings. Every screenshot below is from the current build running on an Android emulator.



Sign in
Demo mode needs no account



Dashboard
Balance, stats, budget window



Quick actions
Five ways to add data

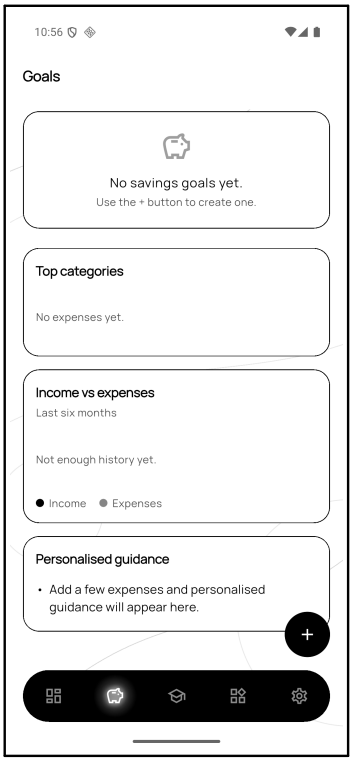
Dashboard

The dashboard is the app's centre of gravity. A manual balance snapshot drives everything below it: four derived statistics, a budgeting window that spreads the remaining balance over a chosen date range, then a stack of cards - smart insights, spending pattern, daily planner, location guidance, subscriptions, event calendar and recent transactions. Every figure on the page is recomputed from the live transaction list, never cached.

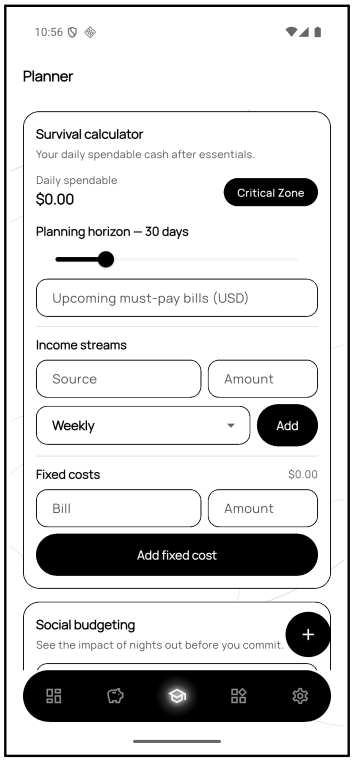
The design language

The interface is deliberately monochrome: black, white and three greys, one geometric typeface, and a single reserved red used only to confirm a destructive action. Every container - card, input, tile, chip - carries a slim ink border, while the rules drawn *inside* a container stay pale. That distinction is what stops a card full of rows reading as a table, and it is enforced by its own test suite rather than by eye. Colour emoji are excluded for the same reason: they render in the platform's own colour font whatever the styling says, so category marks are drawn as line icons instead.

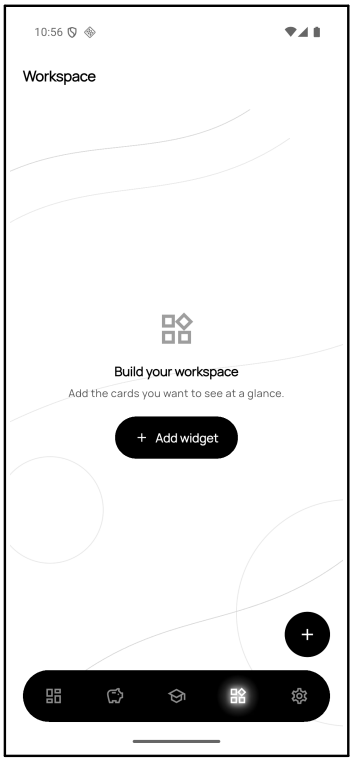
Saving, planning, arranging



Goals
Savings targets and trends



Planner
Survival calculator, runway



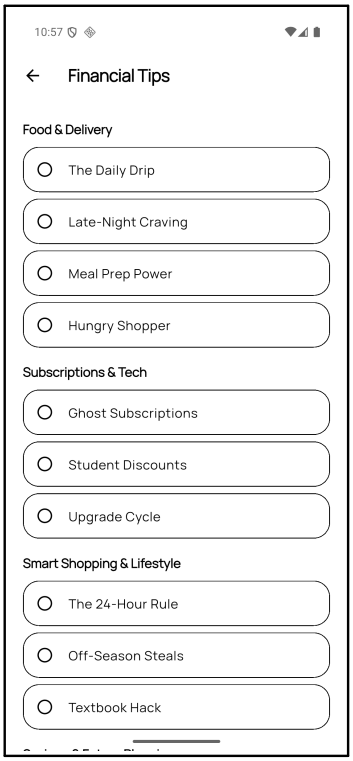
Workspace
16 arrangeable widgets

Goals, Planner and Workspace

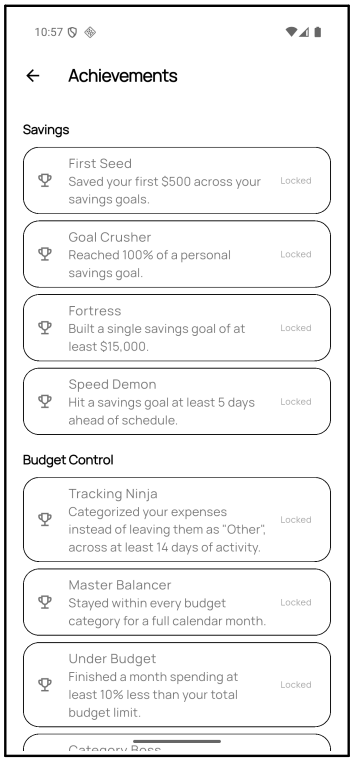
Goals holds savings targets with progress rings, a six-month income-versus-expense chart and narrative guidance derived from the user's two largest spending categories. **Planner** is the forward-looking half: a survival calculator that turns income streams, fixed costs and upcoming bills into a daily spendable figure and a Green / Tight / Critical verdict, plus social-event budgeting and a loan runway visualiser. **Workspace** is a free-form canvas of sixteen widget types the user arranges, resizes and reorders; the layout persists across launches.

All three are empty in these captures because demo mode starts from a clean slate, which is itself a deliberate piece of the design: every empty state says what it is waiting for rather than simply reporting nothing, and no screen invents placeholder data to look busy. An earlier build did fabricate a sample subscription when the user had no transactions, and it read as a bug the moment the first real entry made it disappear.

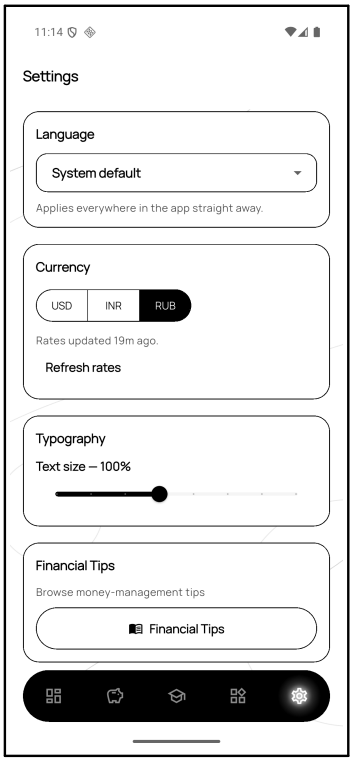
Learning and motivation



Financial Tips
18 cards, 5 categories



Achievements
14 badges, 4 categories



Settings
Language, currency, type size

Two features exist to keep people coming back rather than to compute anything. **Financial Tips** is a library of eighteen short money-management cards grouped into five themes, with read-tracking so the list remembers where the user got to. **Achievements** awards fourteen badges across savings, budget control, habit streaks and a small special category - unlocked by real behaviour such as reaching a goal, categorising expenses consistently for a fortnight, or finishing a month under budget.

Both are deliberately content-free in code: the modules hold only stable identifiers and category groupings, while every title and description lives in the localisation catalogue. Wording can change, and Russian can differ structurally from English, without either module being touched.

Receipt scanning

The most technically involved feature does not have a tab of its own. From the quick-action sheet the user can photograph a receipt or pick several from the gallery at once. Text recognition runs on-device via ML Kit; a parser then extracts the merchant, total, currency and line items, cross-checks the total against the receipt's own arithmetic, and hands back a confidence rating. Nothing is written until the user confirms it on a review screen.

How it is put together

The codebase is organised in three layers with a hard rule between them, which is what makes the interesting half of the product testable without a device.

Layer	Contents	Rule
logic/	9 pure-Dart modules: budget maths, insights, subscription detection, receipt parsing, badges, literacy cards, events, student planner, batch queue.	No Flutter imports, no display strings, no clock. Decides what is true.
state/	11 ChangeNotifier providers - finance, currency, planner, workspace, budgets, achievements, literacy, locale, theme, auth, student planner.	Owns mutation and persistence. One source of truth per domain.
screens/ + widgets/	5 tabs, 4 pushed screens, a shared design-system component set.	Renders. Holds no business rules.
l10n/presenters.dart	The bridge. Turns a logic-layer descriptor into a sentence in the active language.	The reason one rule can read correctly in two languages.

Why the logic layer is kept Flutter-free

A rule expressed as a widget can only be tested by building a widget tree. A rule expressed as a function over plain data can be tested with a list and an expectation. The receipt parser is the clearest case: it takes lines of already-recognised text and returns a structured result, so the whole of its behaviour is exercised against fixture text with no camera, no emulator and no native dependency anywhere in the test.

Start-up and persistence

Every store hydrates from disk *before* the first frame, so the interface never flashes zeroes over real data. Currency is the single exception and is deliberately not awaited, because it may reach the network and the app is perfectly usable on cached or fallback rates.

- Writes are **debounced by 500 ms**. Encoding the finance blob costs roughly nine milliseconds, which is over half a frame at 60 fps, on the UI thread - so a burst of keystrokes must collapse into one write.
- The app **flushes every store on pause, detach and hide**, because Android may kill a backgrounded process at any moment and a debounce window is exactly when data would be lost.
- Aggregates are computed **once per mutation**, not once per widget build. Each dashboard card used to walk the whole transaction list; a single build read four of them.

Four problems worth describing

1. Money was being quantised on the way in

Amounts are stored in USD and converted for display. Originally both directions rounded to two decimals - and the inbound direction is the one that gets *persisted*. One US cent is nearly a rupee, so storing a rounded value quantised the underlying figure, and because the error was a consistent bias rather than noise it accumulated instead of cancelling out. Entering 50,000 rupees returned 49,999.83; one rupee round-tripped to 0.83, a seventeen per cent error.

The fix is one line in each direction: conversion into storage is never rounded, and rounding happens only where a human reads the number. The regression test is written as a *round trip* - enter, store, read back - because a test that only checks the conversion arithmetic passes happily while the round trip is broken.

2. Reading a total off a receipt

A receipt usually states its total more than once: as a labelled line, as cash minus change, and implicitly as the sum of its items. The parser ranks labelled candidates by keyword tier and position, then tries to corroborate the winner against the other two signals. Where the label disagrees with cash minus change, the arithmetic wins - it is arithmetic the receipt performed itself - but the label is still reported, so a user holding the paper can settle it.

- Lines containing cash, change, tender or a card ending are excluded from ever being read as the total, before ranking *and* before the largest-number fallback.
- Item extraction is deliberately conservative: over-collecting would poison the item-sum cross-check, and one fewer signal is better than a wrong one.
- A sideways photograph is retried at 90, 270 and 180 degrees. Recognition returns almost nothing on a rotated image, and that is a far more common real-world failure than genuinely unreadable print.

3. Batch scanning without killing the process

Several receipts can be processed at once, three at a time. The bound is not arbitrary: fully sequential leaves the user watching a spinner, while fully parallel holds ten decoded full-resolution bitmaps in memory simultaneously, which is how a mid-range handset gets its process killed. Dates come from EXIF where the photograph carries them, so a receipt shot on Tuesday is dated Tuesday; duplicates are flagged, failed rows never block readable ones, and nothing is written until the batch is confirmed.

4. One typeface, two alphabets

The Russian translation shipped in an earlier build rendering in whatever substitute font the handset chose, because the typeface in use carried no Cyrillic glyphs at all. Moving to Manrope fixed it. Counted phrases decline properly - one, two and five days take different noun endings in Russian - and amounts follow the currency rather than the interface language, so rupees keep Indian digit grouping whichever language the app is read in.

Testing, security and release

Test strategy

537 tests pass on the current build. The suite is deliberately split by what it is asking, because the two questions fail in different ways.

Suite	Asks	Examples
Logic	Is the rule correct?	Receipt parser fixtures, budget maths, subscription cadence, student planner, currency round trip.
Widget	Does it render without breaking?	All 16 workspace widgets at three sizes, every tab in both languages, at 1.3x text scale.
Design	Is it legible?	Contrast of every control state, no colour emoji, container borders, typography ramp.
Performance	Is it fast enough?	Derived-cost benchmarks at 200, 1,000 and 5,000 transactions; persistence write counting.

The widget sweep exists because larger type is the most reliable way to break a layout. Running every screen in Russian at 1.3x text scale catches overflow that no amount of looking at an English phone at default size ever will.

Security and data handling

- **Session tokens sit in the Android Keystore.** The default storage writes access and refresh tokens as plaintext into shared preferences - readable on a rooted device and included in an unencrypted backup. Any token left behind by an older build is deleted on upgrade rather than migrated: a token that has been sitting in a world-backupable file should not be treated as trustworthy.
- **Logging is stripped from release builds.** Debug printing is not removed automatically and is readable over a cable; for a finance app that means amounts, merchants and email addresses. Messages are passed as closures so even the string interpolation never runs, and failures log an exception's type rather than its text, because exception messages here would carry the stored finance data or the email being signed in.
- **Cleartext traffic is denied at the OS level**, rather than relying on every call site happening to use a secure URL.
- **Backup is disabled** and receipt images stay in app-private storage.
- **Signing keys are kept out of the repository**, with no default that would silently commit a password. Release builds are obfuscated and shrunk.

Release

Current build is **v0.2.0 build 9**, distributed as signed per-architecture APKs of roughly 31 to 37 MB, with install instructions and SHA-256 checksums alongside them.

Who built what

Read this section before circulating

The split below was drafted as a starting point, not recorded from the project history. Please correct it against what each person actually did before this document goes to anyone else - the rest of the report is drawn from the codebase and is accurate as written.

Member	Focus	Contribution
Vansh	Architecture and state	Set the three-layer structure and the rule that the logic layer stays free of Flutter. Built the finance store and its derived aggregates, the debounced persistence and lifecycle flush, and the currency provider - including the storage-rounding fix. Owned the release pipeline: signing, shrinking, obfuscation and APK distribution.
Kintan	Design system and UI	Led the monochrome redesign: the palette, the single type ramp, and the component set of cards, buttons, progress rings, list rows and icon tiles. Built the animated bottom navigation and the decorative backdrop, and drove the contrast and text-scale work that keeps the interface legible at 1.3x.
Athena	Receipt pipeline	Built the receipt parser end to end - keyword tiering, payment-line exclusion, the cash-minus-change and item-sum cross-checks, currency identification and the confidence model. Added the rotation retry and the bounded-concurrency batch queue with EXIF dating and duplicate flagging.
Akshit	Planner and workspace	Built the survival calculator, the Green / Tight / Critical zone model, social-event budgeting and the loan runway visualiser. Implemented the workspace canvas with its sixteen widget types, resizing, reordering and layout persistence.
Meehika	Localisation, engagement, QA	Delivered the complete Russian translation including plural declension, and the presenter layer that separates rules from wording. Built Financial Tips and Achievements with their read-tracking and unlock rules, and owned the widget sweep that runs every screen in both languages at enlarged text.

Shared practice

- Changes reach the main branch through reviewed pull requests from personal forks.
- Analyzer and the full test suite must pass before a review is requested, and behaviour changes are reflected in the specification in the same change.
- The feature-status document is updated in the same change that alters what works, on the principle that a status document which lags the code is worse than none, because people trust it.

Known gaps and what comes next

Stated plainly, because a report that lists only what works is not much use to whoever picks the project up next.

Gap	Status	Why it matters
Transactions cannot be edited or deleted	Not started	The most visible gap, and more pressing since batch scanning landed: a batch can write ten rows at once and the only way to remove a wrong one is a full reset. Highest priority.
Goals and budget categories cannot be deleted	Not started	Same shape as above.
No backend, no sync, no recovery	Deferred by decision	All data is device-local. Clearing app data loses everything, and there is no multi-device story. Accounts are written but unconfigured.
Fixed liabilities	Storage key only	Reserved, with nothing built behind it.
iOS	Not started	Android only; there is no iOS project in the repository.
Continuous integration	Not started	Analyzer, tests and builds all run locally, so the release checks depend on someone remembering to run them.

Next

- Transaction edit and delete, routed through the same persistence path so every derived figure stays correct.
- Continuous integration, so the analyzer and the suite stop being a matter of discipline.
- A decision on the backend - finishing the hosted-account path or building a service - after which sync and recovery become possible.

Cash Compass v0.2.0 build 9. Figures in this report were taken from the codebase and the passing test suite at the time of writing; screenshots are from the current build running on an Android emulator.